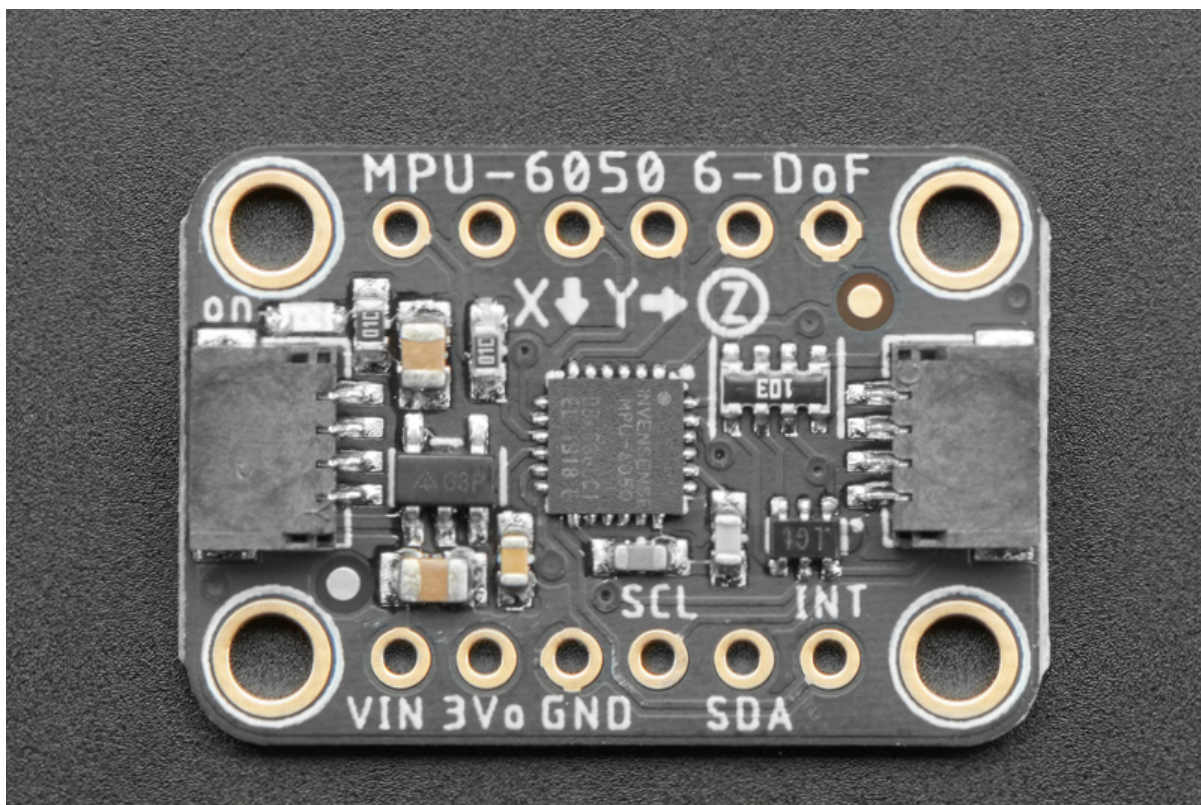




MPU6050 6-DoF Accelerometer and Gyro

Created by Bryan Siepert



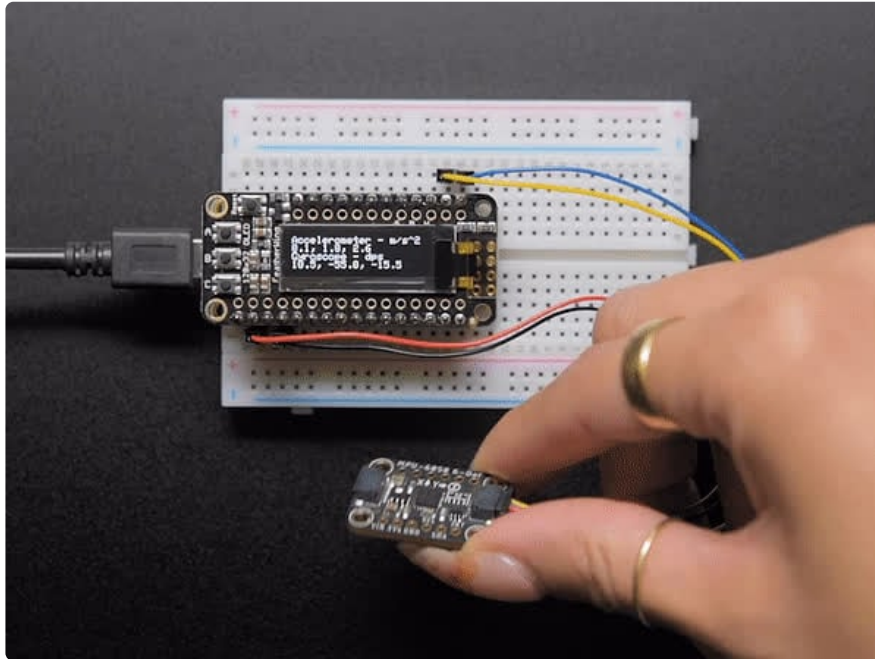
<https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro>

Last updated on 2026-09-30 05:01:59 PM UTC

Table of Contents

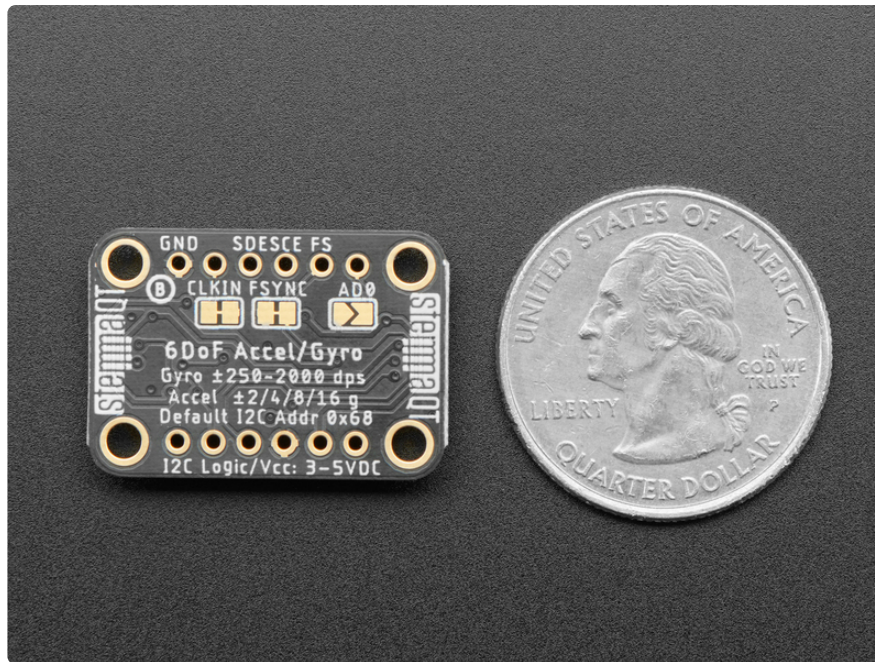
Overview	3
Pinouts	5
• Power Pins • I2C Logic Pins • Other Pins	
Arduino	6
• Wiring • Library Installation • Basic Reading Example • Basic Readings Example Code	
Arduino Docs	10
Python and CircuitPython	10
• CircuitPython Microcontroller Wiring • Python Computer Wiring • CircuitPython Installation of MPU6050 Library • Python Installation of MPU6050 Library • CircuitPython & Python Usage • Example Code	
Python Docs	15
Downloads	15
• Files • Schematic • Fab Print	

Overview

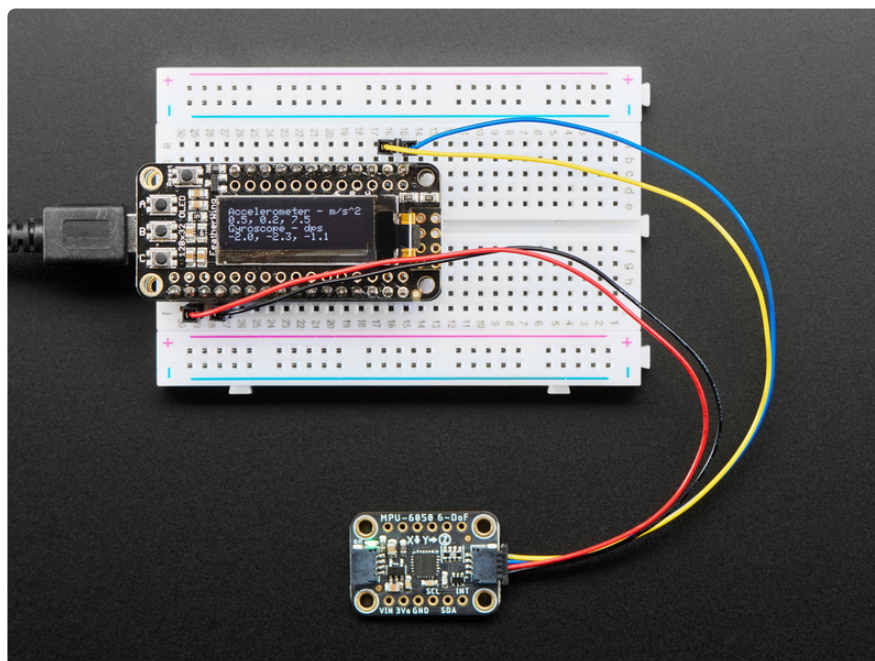


The MPU-6050 is a popular six DoF accelerometer and gyroscope (gyro) that has all the info you need on how things are shakin' and spinnin' . With six axes of sensing and 16-bit measurements, you'll have everything you need to give your robot friend a sense of balance, using the MPU-6050 as its inner ear.

This combination of gyroscopes and accelerometers is commonly referred to as an Inertial **M**easurement **U**nit or **IMU**. Not so long ago IMUs were the [size of a breadbox](#) () and cost upwards of **\$50,000**! While you're not going to be using it to guide your mars rocket, the MPU-6050 is several orders of magnitude smaller and a bargain at a price three orders of magnitude less! Now you can add telemetry to your [water rocket](#) () (with some waterproofing).

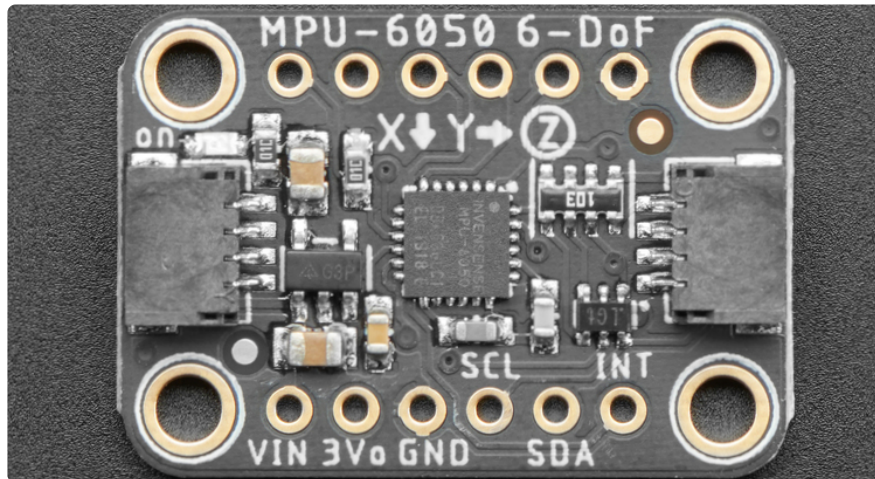


As with all Adafruit breakouts, we've done the work to make the MPU-6050 super easy to use. We've put it on a breakout board with the required support circuitry and connectors to make it easy to work with. And of course we've added [SparkFun Qwiic \(\)](#) compatible [STEMMA QT \(\)](#) JST SH connectors that allow you to get going **without needing to solder**. Just use a [STEMMA QT adapter cable](#) (<http://adafru.it/4209>), plug it into your favorite micro or Blinka supported SBC and you're ready to rock!



"What is a sensor without a driver?" you might say. To that I would reply "Who cares, we got some for you right here". Be it Arduino, CircuitPython or Python on a computer (single board or even [multi-board! \(\)](#)), we've got you covered.

Pinouts



Power Pins

- **Vin** - this is the power pin. Since the sensor chip uses 3 VDC, we have included a voltage regulator on board that will take 3-5VDC and safely convert it down. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V microcontroller like Arduino, use 5V
- **3Vo** - this is the 3.3V output from the voltage regulator, you can grab up to 100mA from this if you like
- **GND** - common ground for power and logic

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller's I2C clock line. This pin is level shifted so you can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** -I2C data pin, connect to your microcontroller's I2C data line. This pin is level shifted so you can use 3-5V logic, and there's a **10K pullup** on this pin.
- [STEMMA QT \(\)](#) - These connectors allow you to connect to dev boards with **STEMMA QT** connectors or to other things with [various associated accessories \(\)](#)

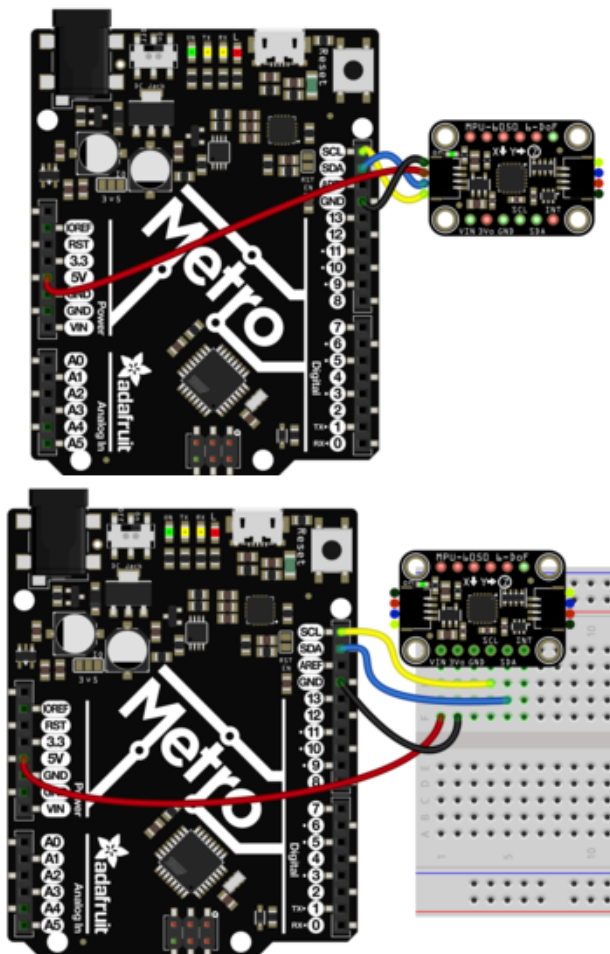
Other Pins

- **INT** - This is the interrupt pin. You can setup the MPU-6050 to pull this low when certain conditions are met such as new measurement data being available. Consult the [datasheet and register map \(\)](#) for usage
- **AD0** - I2C Address pin. Pulling this pin high or bridging the solder jumper on the back will change the I2C address from **0x68** to **0x69**
- **FS, SCE, SDE, CLKIN** - Pins for advanced users to connect the MPU-6050 to another sensor. Consult the [datasheet and register map \(\)](#) for usage

Arduino

Wiring

Wiring the MPU-6050 to communicate with your microcontroller is straight forward thanks to the I2C interface. For these examples we can use the Metro or Arduino to take measurements. The instructions below show a [Metro](http://adafru.it/2488) (<http://adafru.it/2488>), but the same applies to an Arduino



Connect **board VCC** (red wire) to **Arduino 5V** if you are running a **5V** board Arduino (Uno, etc.). If your board is **3V**, connect to that instead.

Connect **board GND** (black wire) to **Arduino GND**

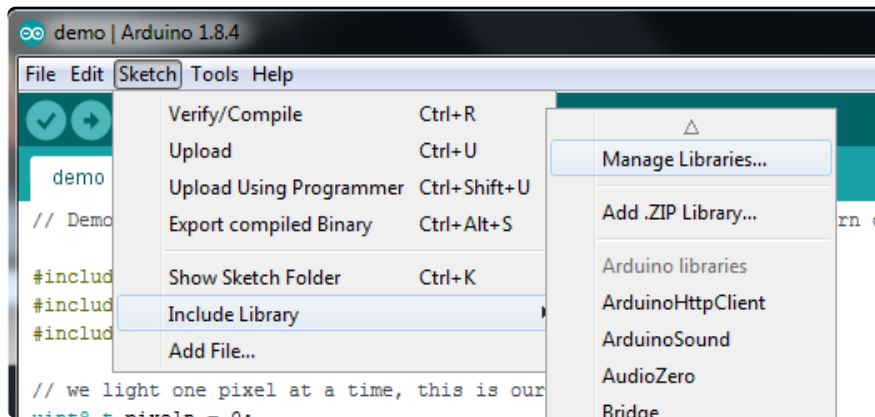
Connect **board SCL** (yellow wire) to **Arduino SCL**

Connect **board SDA** (blue wire) to **Arduino SDA**

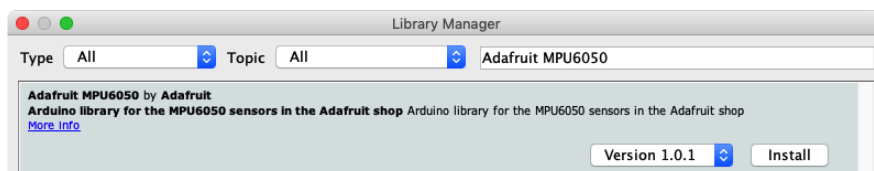
Library Installation

Once wired up, to start using the MPU-6050 you'll need to install the [Adafruit MPU6050 library](#) (). The library is available through the Arduino library manager so we recommend taking that approach.

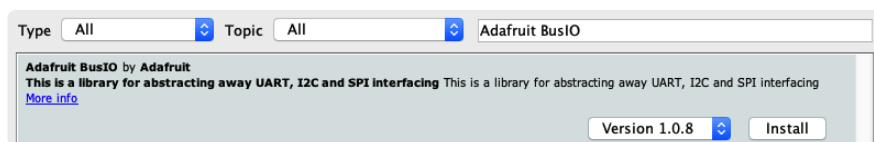
From the Arduino IDE, open up the Library Manager:



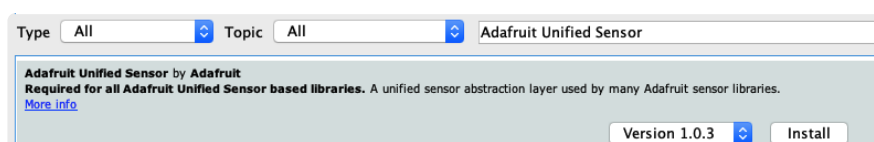
Click the **Manage Libraries ...** menu item, search for **Adafruit MPU6050**, and select the **Adafruit MPU6050** library and click **Install**:



Then follow the same process for the **Adafruit BusIO** library.



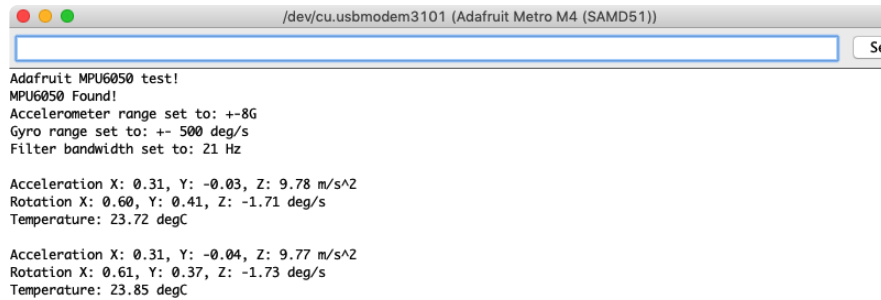
Lastly find and install the **Adafruit Unified Sensor** library



Basic Reading Example

Open up **File -> Examples -> Adafruit MPU6050 -> basic_readings** and upload to your Arduino wired up to the sensor.

Once you've uploaded the sketch to your board open up the Serial Monitor (**Tools -> Serial Monitor**) at **115200 baud**. You should see the acceleration, rotation measurements, and temperature being printed like so:



Give the sensor a wiggle or a spin and watch how the measurements change!

Basic Readings Example Code

```
1 // Basic demo for accelerometer readings from Adafruit MPU6050
2
3 #include <Adafruit_MPU6050.h>
4 #include <Adafruit_Sensor.h>
5 #include <Wire.h>
6
7 Adafruit_MPU6050 mpu;
8
9 void setup(void) {
10   Serial.begin(115200);
11   while (!Serial)
12     delay(10); // will pause Zero, Leonardo, etc until serial console opens
13
14   Serial.println("Adafruit MPU6050 test!");
15
16   // Try to initialize!
17   if (!mpu.begin()) {
18     Serial.println("Failed to find MPU6050 chip");
19     while (1) {
20       delay(10);
21     }
22   }
23   Serial.println("MPU6050 Found!");
24
25   mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
26   Serial.print("Accelerometer range set to: ");
```

```

27  switch (mpu.getAccelerometerRange()) {
28  case MPU6050_RANGE_2_G:
29      Serial.println("+2G");
30      break;
31  case MPU6050_RANGE_4_G:
32      Serial.println("+4G");
33      break;
34  case MPU6050_RANGE_8_G:
35      Serial.println("+8G");
36      break;
37  case MPU6050_RANGE_16_G:
38      Serial.println("+16G");
39      break;
40  }
41  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
42  Serial.print("Gyro range set to: ");
43  switch (mpu.getGyroRange()) {
44  case MPU6050_RANGE_250_DEG:
45      Serial.println("+ 250 deg/s");
46      break;
47  case MPU6050_RANGE_500_DEG:
48      Serial.println("+ 500 deg/s");
49      break;
50  case MPU6050_RANGE_1000_DEG:
51      Serial.println("+ 1000 deg/s");
52      break;
53  case MPU6050_RANGE_2000_DEG:
54      Serial.println("+ 2000 deg/s");
55      break;
56  }
57
58  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
59  Serial.print("Filter bandwidth set to: ");
60  switch (mpu.getFilterBandwidth()) {
61  case MPU6050_BAND_260_HZ:
62      Serial.println("260 Hz");
63      break;
64  case MPU6050_BAND_184_HZ:
65      Serial.println("184 Hz");
66      break;
67  case MPU6050_BAND_94_HZ:
68      Serial.println("94 Hz");
69      break;
70  case MPU6050_BAND_44_HZ:
71      Serial.println("44 Hz");
72      break;
73  case MPU6050_BAND_21_HZ:
74      Serial.println("21 Hz");
75      break;
76  case MPU6050_BAND_10_HZ:
77      Serial.println("10 Hz");
78      break;
79  case MPU6050_BAND_5_HZ:
80      Serial.println("5 Hz");
81      break;
82  }

```

```

83
84     Serial.println("");
85     delay(100);
86 }
87
88 void loop() {
89
90     /* Get new sensor events with the readings */
91     sensors_event_t a, g, temp;
92     mpu.getEvent(&a, &g, &temp);
93
94     /* Print out the values */
95     Serial.print("Acceleration X: ");
96     Serial.print(a.acceleration.x);
97     Serial.print(", Y: ");
98     Serial.print(a.acceleration.y);
99     Serial.print(", Z: ");
100    Serial.print(a.acceleration.z);
101    Serial.println(" m/s^2");
102
103    Serial.print("Rotation X: ");
104    Serial.print(g.gyro.x);
105    Serial.print(", Y: ");
106    Serial.print(g.gyro.y);
107    Serial.print(", Z: ");
108    Serial.print(g.gyro.z);
109    Serial.println(" rad/s");
110
111    Serial.print("Temperature: ");
112    Serial.print(temp.temperature);
113    Serial.println(" degC");
114
115    Serial.println("");
116    delay(500);
117 }

```

https://github.com/adafruit/Adafruit_MPU6050/blob/master/examples/basic_readings/basic_readings.ino

Arduino Docs

[Arduino Docs \(\)](#)

Python and CircuitPython

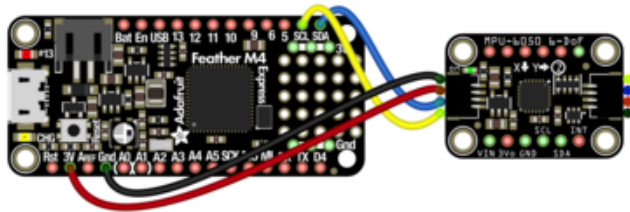
It's easy to use the MPU6050 sensor with CircuitPython and the [Adafruit CircuitPython MPU6050 \(\)](#) library. This library allows you to easily write Python code that reads the acceleration and adjust the measurement settings.

You can use this sensor with any CircuitPython microcontroller board or with a Linux

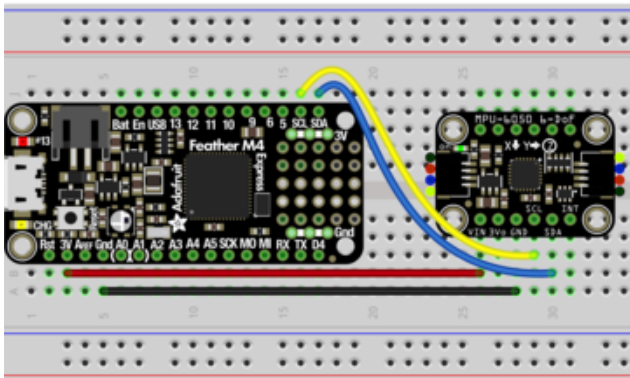
single board computer that has GPIO and Python [thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library](#) ().

CircuitPython Microcontroller Wiring

First wire up a MPU6050 to your board exactly as follows. Here is an example of the MPU6050 wired to a [Feather](http://adafru.it/3857) (<http://adafru.it/3857>) using I2C:



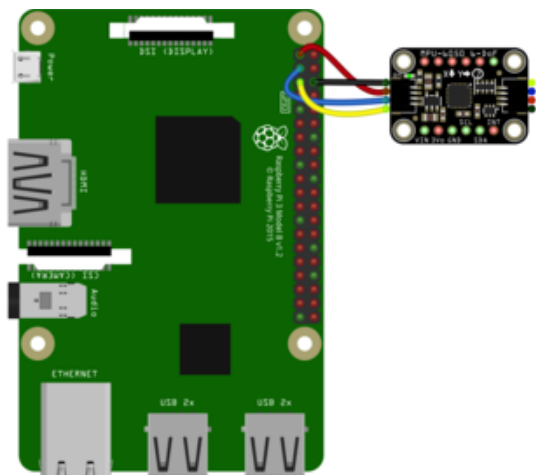
Board 3V to sensor VIN (red wire)
Board GND to sensor GND (black wire)
Board SCL to sensor SCL (yellow wire)
Board SDA to sensor SDA (blue wire)



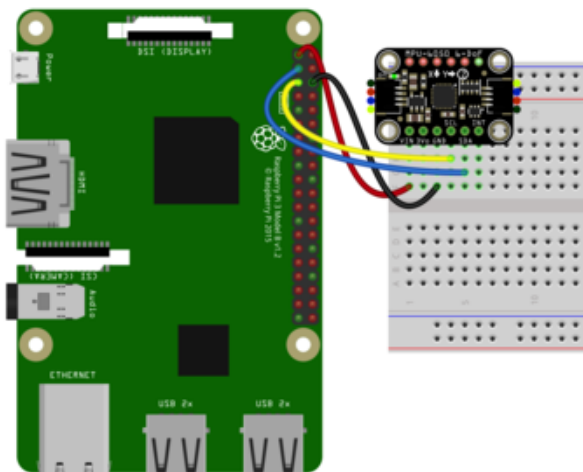
Python Computer Wiring

Since there's dozens of Linux computers/boards you can use we will show wiring for [Raspberry Pi](http://adafru.it/3055) (<http://adafru.it/3055>). For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](#) ().

Here's the Raspberry Pi wired with I2C:



Pi 3V to sensor VCC (red wire)
Pi GND to sensor GND (black wire)
Pi SCL to sensor SCL (yellow wire)
Pi SDA to sensor SDA (blue wire)



CircuitPython Installation of MPU6050 Library

You'll need to install the [Adafruit CircuitPython MPU6050 \(\)](#) library on your CircuitPython board.

First make sure you are running the [latest version of Adafruit CircuitPython \(\)](#) for your board.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle \(\)](#). Our CircuitPython starter guide has [a great page on how to install the library bundle \(\)](#).

For non-express boards like the Trinket M0 or Gemma M0, you'll need to manually install the necessary libraries from the bundle:

- `adafruit_mpu6050.mpy`
- `adafruit_bus_device`
- `adafruit_register`

Before continuing make sure your board's **lib** folder or root filesystem has the **adafruit_mpu6050.mpy**, **adafruit_bus_device**, and **adafruit_register** files and folders copied over.

Next [connect to the board's serial REPL](#) () so you are at the CircuitPython `>>>` prompt.

Python Installation of MPU6050 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. [Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready](#) ()!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-mpu6050`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython & Python Usage

To demonstrate the usage of the sensor we'll initialize it and read the acceleration, rotation, and temperature measurements from the board's Python REPL.

Run the following code to import the necessary modules and initialize the I2C connection with the sensor:

```
1 import time
2 import board
3 import adafruit_mpu6050
4
5 i2c = board.I2C()
6 mpu = adafruit_mpu6050.MPU6050(i2c)
```

```
>>> import time
>>> import board
>>> import adafruit_mpu6050
>>> i2c = board.I2C()
>>> mpu = adafruit_mpu6050.MPU6050(i2c)
```

Now you're ready to read values from the sensor using these properties:

- **acceleration** - The acceleration forces in the X, Y, and Z axes in m/s^2
- **gyro** - The rotation measurement on the X, Y, and Z axes in radians/sec
- **temperature** - The temperature of the sensor in degrees C

For example, to print out the acceleration, gyro, and temperature values:

```
>>> print("Acceleration: X: %.2f, Y: %.2f, Z: %.2f m/s^2"%(mpu.acceleration))
Acceleration: X: 0.39, Y: 0.00, Z: 9.87 m/s^2
>>> print("Gyro X: %.2f, Y: %.2f, Z: %.2f degrees/s"%(mpu.gyro))
Gyro X: 0.58, Y: 0.33, Z: -1.85 degrees/s
>>> print("Temperature: %.2f C"%mpu.temperature)
Temperature: 24.98 C
>>>
```

That's all there is to it! Go forth and imbue your robot friends with the gift of being able to maybe not fall down all the time. Maybe.

Example Code

```
1 # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2 # SPDX-License-Identifier: MIT
3
4 import time
5
6 import board
7
8 import adafruit_mpu6050
9
10 i2c = board.I2C() # uses board.SCL and board.SDA
11 # i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a microcontroller
12 mpu = adafruit_mpu6050.MPU6050(i2c)
13
14 while True:
15     print(
16         f"Acceleration: X:{mpu.acceleration[0]:.2f}, Y: {mpu.acceleration[1]:.2f}, Z: {mpu.acceleration[2]:.2f} m/
17         s^2" # noqa: E501
18     )
19     print(f"Gyro X:{mpu.gyro[0]:.2f}, Y: {mpu.gyro[1]:.2f}, Z: {mpu.gyro[2]:.2f} rad/s")
20     print(f"Temperature: {mpu.temperature:.2f} C")
21     print("")
22     time.sleep(1)
```

https://github.com/adafruit/Adafruit_CircuitPython_MPU6050/blob/main/examples/mpu6050_simpletest.py

Python Docs

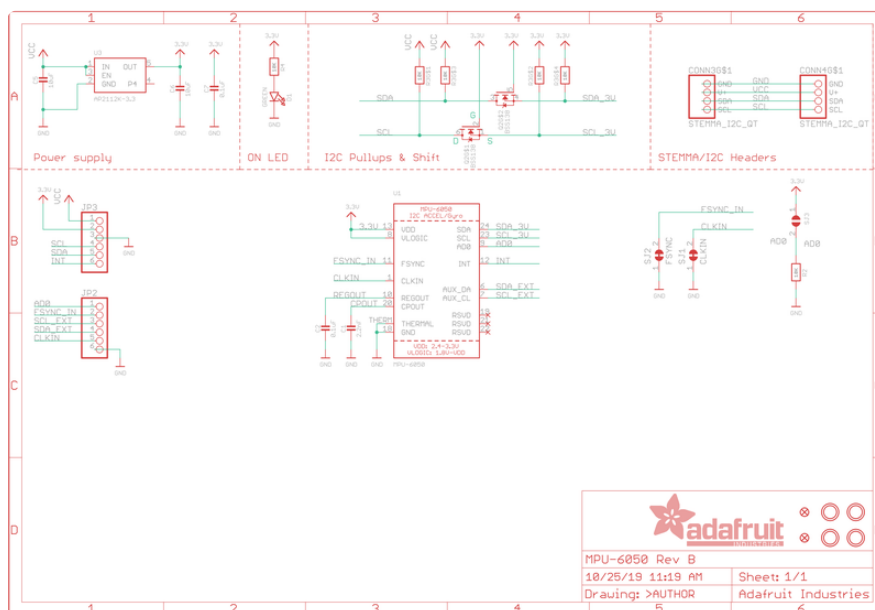
[Python Docs \(\)](#)

Downloads

Files

- [MPU6050 Datasheet \(\)](#)
- [MPU6050 Register Map \(\)](#)
- [EagleCAD files on GitHub \(\)](#)
- [Fritzing object in Adafruit Fritzing Library \(\)](#)

Schematic



Fab Print

